Homework 6: Graphs

Guideline

Homework chatbot. Open the homework Gem here. Use this Gem when working through the AI-related sections. The bot is already curated with the right answers and hints to provide the right level of guidance to help you learn.

Showing your honest attempt can only help. There is no advantage in cheating. I give 100% credit on attempts, even if they are incorrect. Cheating on homework will also only hurt you. If you cheat, I won't be able to give you the proper feedback to help you learn, nor adjust the difficulty of the quizzes. You will also likely fail on the in-person mastery quizzes, which test concepts tackled in the homework.

Collaboration: Only after solo attempt. You may collaborate and use external resources, but (1) you are encouraged to do the homework **alone first** and (2) you must **attribute** these external resources when you do use them.

AI: Follow guideline. The problems will be very clear on when you are allowed to use AI. Please follow them. Here is why: (1) Unregulated usage of AI harms learning (2) Even in the age of AI, your AI-independent ability matters. The **only** AI tool you should use on this homework is the homework-specific Gemini Gem that I have created for you.

Where AI is allowed on this homework. Problems **1 and 2 are entirely AI-free** -- no Gem, no chatbot, nothing. In problems 3 and 4, every part **before** an AI-check step is solo: work out the representation, the pseudocode and the code on your own first, then bring what you wrote to the Gem. The whole point of the AI steps is to check work you have already done.

Submission. Upload all files to the Homework 6 assignment on Canvas. **5 files total:**

1. **One answer document** containing your answers to every question below. Name it `lastname-hw6.pdf` (or `.txt`). LaTeX isn't required -- plain text, Google Docs → Download as PDF, or Word → Save as PDF are all fine.
2. **Four AI chat transcript PDFs**, one for each AI-check step. Start a **new chat** in the Gem for each step, so that each transcript covers only that step:
   - `pacific-c.pdf`, `pacific-e.pdf`
   - `accounts-d.pdf`, `accounts-f.pdf`

Accepted file formats: `.pdf` for everything; `.txt` is also fine for the answer document. Do **not** paste chat content into your answer document, and do **not** submit a link to a Google Doc -- upload the file itself.

Due: Sunday, November 8, 2026 at 11:59pm. See the syllabus for the late-day policy.

Points: 22 total. Remember, I give full points to every question that was attempted, and 0 only for a question left blank. What matters is that you have **thought** through the problems enough to prepare yourself for the quizzes.

1. Course schedule (3 pts total)

There are `numCourses` courses labeled `0` to `numCourses - 1`. You are given a list `prerequisites`, where `prerequisites[i] = [a, b]` means **you must take course `b` before course `a`**.

Return `true` if you can finish every course, and `false` otherwise.

Leetcode link

Here are some example input and output:

`prerequisites`	answer	why
`[[1,0]]`	`true`	take 0, then 1
`[[1,0],[0,1]]`	`false`	each one needs the other first, so neither can ever start
`[[1,0],[2,1],[3,2]]`	`true`	one long chain: 0, 1, 2, 3

This one is a straightforward application of what we did in lecture. You already have every piece; the only work is recognizing which piece.

No AI on this problem at all.

(a) Representation (1 pt)

What are the **vertices** and the **edges** of the graph you would build here? Be concrete: say what a single vertex is, and say exactly when two vertices get an edge between them -- including which way the edge points. Answer in 3 to 5 sentences.

(b) Graph algorithm (1 pt)

Which graph algorithm from lecture solves this, once you have that graph? Answer in **one sentence**.

(c) Code (1 pt)

Code it up on Leetcode. Turn in your final code, pasted into your answer document, and paste your Leetcode submission URL showing a successful submission.

Do this **without AI**. You may look at the lecture slides while you learn it -- but then close them and redo it from scratch, and keep redoing it until you can write it with no reference at all. Imagine you are at a whiteboard in an interview.

One note on the lecture template: it keeps `state` and `cycle_found` as module-level variables so the slide stays readable. On Leetcode your code lives inside a `Solution` class, so make them `self.state` and `self.cycle_found` instead -- otherwise assigning to them inside a helper creates a fresh local and your changes vanish.

2. 0-1 Matrix (3 pts total)

You are given an `R x C` binary matrix `mat`. For **every** cell, return the distance to the nearest `0`. Two cells are adjacent if they share an edge in one of these 4 directions: up, down, left or right.

Leetcode link

Here is an example.

Input:

	`c=0`	`c=1`	`c=2`
`r=0`	0	0	0
`r=1`	0	1	0
`r=2`	1	1	1

Output:

	`c=0`	`c=1`	`c=2`
`r=0`	0	0	0
`r=1`	0	1	0
`r=2`	1	2	1

Why, for two of the output cells:

- `(r=1, c=1)` is a 1, because it's one step away from a 0 (it's surrounded by three 0's).
- `(r=2, c=1)` is a 2, because you need 2 steps to get to the nearest 0, e.g., by going up then left.

No AI on this problem at all.

(a) Representation (1 pt)

What are the **vertices** and the **edges**? Be concrete: what is a single vertex, and when do two vertices get an edge? Answer in 3 to 5 sentences.

(b) Graph algorithm (1 pt)

Which graph algorithm solves this? Answer in **one sentence**.

(c) Pseudocode (1 pt)

You will **not** have to code this one -- you get plenty of implementation practice in problems 1, 3 and 4.

Two things to write down:

1. Which **lecture problem** is this most similar to? Name it, and say in a sentence or two what the two problems have in common.

2. Your **pseudocode** for solving it. A numbered list is fine. Be specific about how you figure out the number to assign to each output cell. At most 5 lines is enough.

3. Pacific Atlantic Water Flow (8 pts total)

You are given an `R x C` grid `heights`, where `heights[r][c]` is the height of that square of land.

The **Pacific** ocean touches the grid's **top** and **left** edges. The **Atlantic** ocean touches the **bottom** and **right** edges.

Rain falls on every square. Water flows from a square to a neighboring square (up, down, left or right) only if the neighbor's height is **less than or equal to** the current square's height. Water can also flow off the edge into whichever ocean that edge touches.

Return a list of all coordinates `[r, c]` from which rainwater can reach **both** oceans.

Leetcode link

Look at the pictures on the Leetcode page -- this one is much easier to see than to read. They show the grid with both oceans marked, which makes the two-ocean condition obvious.

(a) Hint (1 pt)

A naive solution would be to BFS/DFS from every cell and see if Pacific ocean is reachable from that cell, but that would require many graph traversals.

Can you figure out all the cells that can reach the Pacific ocean with just **one** traversal? Describe your idea in 1-3 sentences.

(b) Pseudocode (1 pt)

How will you solve the original problem? Write your plan as pseudocode -- a numbered list is fine. Do **not** use AI. Set a timer for **15 minutes** to simulate you thinking for an interview. This is **tricky.**

(c) AI plan check

Bring your previous answers to the homework chatbot and ask it to check your plan. Here are some example prompts:

- **Verifying.** "Here is the problem (copy paste). Here is my representation and my pseudocode (copy paste). Is this right, and is it efficient?"
- **Brainstorming.** "I have a plan but I think it is slow. Can you give me a hint about the traversal direction?"

(i) Chat transcript (1 pt)

Turn in the chat transcript as a PDF. Name it `pacific-c.pdf`. Do **not** copy paste the chat content into your answer document.

(ii) Updated pseudocode (1 pt)

Write your pseudocode again in your answer document, updated after the conversation. If nothing changed, say so and say why you are confident.

(d) Code (1 pt)

Code it up on Leetcode. Turn in your final code, pasted into your answer document, and paste your Leetcode submission URL showing a successful submission.

Start a stop watch and work **without AI** for the first **30 minutes** -- pretend this is a live interview session. If you have exceeded 30 minutes, you **may** use the homework Gem to help you get to a successful submission.

(e) AI code check

Copy paste your code to the homework gem and ask for a review. Here is an example prompt: "Here is the problem statement (copy) and here is my code (copy). Q1. Can you evaluate for correctness? Q2. Can you help improve my code style?"

(i) Chat transcript (1 pt)

Turn in the chat transcript PDF. Name it `pacific-e.pdf`. Do **not** copy paste the chat content into your answer document.

(ii) Updated code (1 pt)

Paste your final code again, updated after the review. If you changed nothing, say so and say why.

(f) Analysis (1 pt)

Give the big-O **runtime** of your solution, and explain **why** in 3 to 5 sentences. Say what `R` and `C` stand for, and account for every traversal you run.

4. Accounts merge (8 pts total)

You are given a list `accounts`, where `accounts[i][0]` is a person's **name** and the rest of `accounts[i]` are that person's **email addresses**.

Two accounts belong to the same person if they share **at least one email address**. Note that the same name does **not** mean the same person -- two different people can both be called John.

Merge the accounts. For each merged person, return their name followed by all of their emails **sorted alphabetically**.

Leetcode link

Here are some example input and output.

Example 1 -- input:

```
[["John", "johnsmith@mail.com", "john_newyork@mail.com"],
 ["John", "johnsmith@mail.com", "john00@mail.com"],
 ["Mary", "mary@mail.com"],
 ["John", "johnnybravo@mail.com"]]
```

Example 1 -- output:

```
[["John", "john00@mail.com", "john_newyork@mail.com", "johnsmith@mail.com"],
 ["John", "johnnybravo@mail.com"],
 ["Mary", "mary@mail.com"]]
```

Example 1 -- why: the first two Johns both list `johnsmith@mail.com`, so they are the same person and their emails merge into one sorted list. The third John shares no email with anybody, so he stays on his own. Mary is untouched.

Example 2 -- input:

```
[["John", "j1@m.com"],
 ["John", "j2@m.com"]]
```

Example 2 -- output:

```
[["John", "j1@m.com"],
 ["John", "j2@m.com"]]
```

Example 2 -- why: nothing merges. These two accounts have the same name but share no email address, so they are two different people who happen to both be called John.

(a) Representation (1 pt)

Suppose that each account (that is, a single input row) maps to exactly 1 node, and there are no more extra nodes, then **Q:** What are the edges? That is, how do we figure out the adjacent nodes for each node?

(b) Graph algorithm (1 pt)

Which graph algorithm will you use? Answer in **one sentence**.

(c) Pseudocode (1 pt)

How will you solve the original problem? Write your plan as pseudocode. Tackle the following:

- How will you construct the graph's node and edges?
- When will you invoke the graph algorithm?
- How will you use the graph algorithm's output and reformat it back to the original problem's output?

Do **not** use AI. Set a timer for **15 minutes** to simulate you thinking for an interview.

(d) AI plan check

Bring your answers from (a), (b) and (c) to the homework chatbot and ask it to check your plan. Here are some example prompts:

- **Verifying.** "Here is the problem (copy paste). Here is my representation and my pseudocode (copy paste). Is this right?"
- **Brainstorming.** "I cannot decide what my vertices should be. Can you give me a hint?"
- **Asking for clarification.** "How do I get the name back once I have grouped the emails?"

(i) Chat transcript (1 pt)

Turn in the chat transcript as a PDF. Name it `accounts-d.pdf`. Do **not** copy paste the chat content into your answer document.

(ii) Updated pseudocode (1 pt)

Write your pseudocode again in your answer document, updated after the conversation. If nothing changed, say so and say why you are confident.

(e) Code (1 pt)

Code it up on Leetcode, **without AI**. Turn in your final code, pasted into your answer document, and paste your Leetcode submission URL showing a successful submission.

(f) AI code check

Copy paste your code to the homework gem and ask for a review. Here is an example prompt: "Here is the problem statement (copy) and here is my code (copy). Q1. Can you evaluate for correctness? Q2. Can you help improve my code style?"

(i) Chat transcript (1 pt)

Turn in the chat transcript PDF. Name it `accounts-f.pdf`. Do **not** copy paste the chat content into your answer document.

(ii) Updated code (1 pt)

Paste your final code again, updated after the review. If you changed nothing, say so and say why.